FINTECHATHON 2026 · INTERNATIONAL AI TRACK · TOPIC F, AGENT WALLET

Give your AI a card, not your wallet.

ChainBrain gives each AI agent its own card: a smart contract on FISCO BCOS that holds Mock e-HKD and the owner's spending rules. The agent finds sellers and pays them. The card checks every payment against the owner's rules at the moment the money moves, so a fooled agent still can't pay outside them.

8
checks the card runs before any money moves

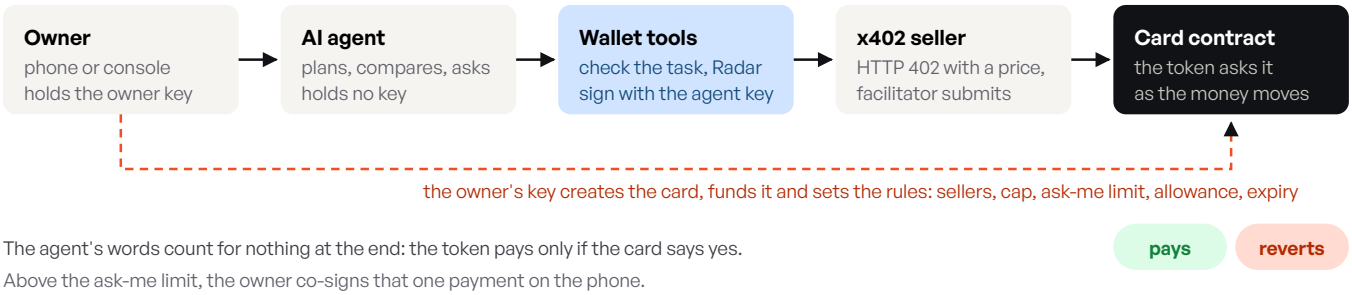
4
Solidity contracts, 356 lines in total

7
agent tools; only **pay** moves money, and none takes an address

462
automated tests, all passing on 19 Oct

20200
chain id of the hosted FISCO BCOS 3.16.4 chain

One payment, start to finish



The agent's words count for nothing at the end: the token pays only if the card says yes.
Above the ask-me limit, the owner co-signs that one payment on the phone.

pays reverts

Each agent, its own card

An agent gets a card and a key that can only sign for that card. A mistake or an attack is limited to that card's balance and rules, never the owner's wallet.

The chain has the last word

Seller allowlist, cap, co-sign limit, weekly allowance, expiry, revoke and freeze live in the card. Code in the server and app checks first, but nothing relies on it.

The owner's key stays on the device

The phone keeps it in the iOS Keychain or Android Keystore; the console keeps it in the browser, optionally behind a passkey. The server never sees it.

Contents

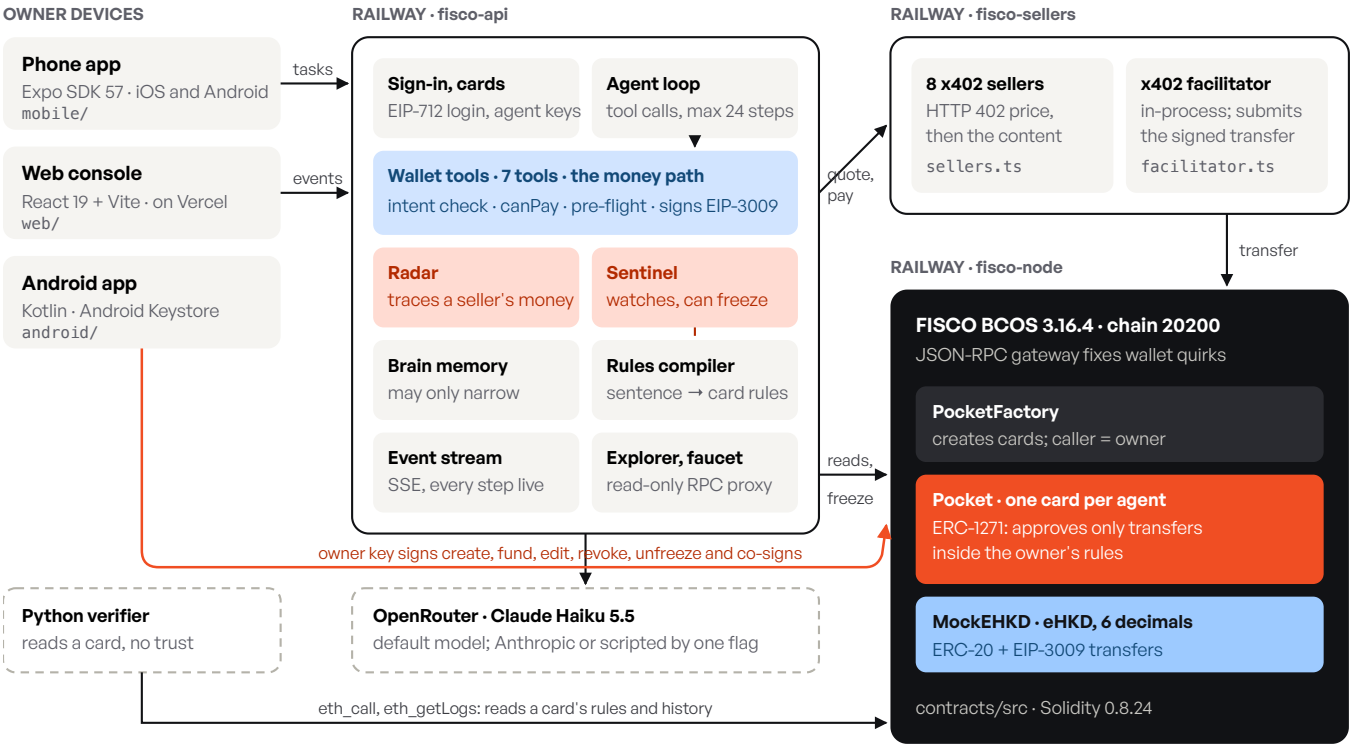
Architecture 2
Keys and sign-in 3
One payment, step by step 4
The card contract 5
Wallet server 6
Agent and AI brain 7
Security and monitoring 8
The apps 9
Running modes 10
Chain and deployment 11
Tests, setup and limits 12

Try it

Console: fintechathon-lian-ai-nao.vercel.app
Explorer: fisco-api-production-85c5.up.railway.app/explorer
Source: github.com/ixgnoy/Fintechathon-LianAiNao
Attack results: ChainBrain-security-test-report.pdf, 63 attacks, HK$0 lost

Architecture

Three owner apps, three hosted services on Railway, and three contracts on one FISCO BCOS chain. Each box below is real, running code; the folder is where it lives in the repository.



Part	What it does	Where it runs · folder
Phone app	Owner's main app: cards, chat with the agent, co-sign with Face ID, Brain graph, ledger	iOS, Android · `mobile/`
Web console	Judges' and presenter's view: live trace, Brain, explorer, evidence, attack arena, faucet	Vercel · `web/`
Wallet API	Sign-in, agent loop, wallet tools, Radar, Sentinel, memory, routines, events, explorer	Railway fisco-api · `server/src`
Sellers	8 x402 sellers that answer HTTP 402 with a price, and the facilitator that submits payments	Railway fisco-sellers · `server/src`
Chain node	FISCO BCOS with a JSON-RPC gateway (fee history, log windows, batch splits, funding)	Railway fisco-node · `deploy/fisco-node`
Contracts	Pocket (the card), PocketFactory, MockEHKD, Clock (FISCO's millisecond time to seconds)	On chain · `contracts/src`
Shared code	Signing guard, EIP-712 login, the built-in demo network used by both apps	In the apps · `shared/`
Verifier	Standard-library Python; checks a card's rules and history against the chain, read-only	Anywhere · `verify/`

Who holds which key

Every key can do one narrow job. The owner's key is the only one that changes a rule or moves money out freely, and it never leaves the owner's device.

Key	Where it lives	Can	Cannot
Owner	Phone: iOS Keychain or Android Keystore. Console: browser, optionally behind a passkey	Create, fund and edit cards; revoke, unfreeze, take funds back; co-sign one payment	Be read by the server: it never leaves the device
Agent, one per card	Derived on the server when used: `keccak256(AGENT_MASTER_KEY ‖ cardId)`, never stored	Sign an exact EIP-3009 transfer from its own card to a quoted seller	Edit a rule, pay a seller off the list, pass the cap, the allowance or the balance, or sign for another person's card
Sentinel (guardian)	Server, `SENTINEL_PK`; fixed in the factory	Freeze a card, with a reason	Move money, unfreeze, or change any rule
Facilitator	Sellers service, `FACILITATOR_PK`	Submit a transfer the agent already signed	Spend from a card without the agent's signature
Issuer	Server, `OWNER_PK`	Mint test Mock e-HKD for the faucet and new visitors	Touch any card's rules or balance

If a key is stolen

agent key
Pays only allowed sellers, under the cap and the allowance. Above the ask-me limit it still needs the owner.

the whole server
Same limits, for every card. The checks in the server are skipped; the card's checks are not.

guardian key
Can freeze cards. Freezing stops payments; the owner unfreezes. No money moves.

owner key
Full control of that owner's cards. That's why it stays in the device's secure storage.

Signing in without a password

1 **Challenge** — single-use 32-byte nonce, 5 minutes

→ 2 **Device signs Login** — EIP-712; the statement says it moves no money

→ 3 **Server checks** — account = hash of the owner address; same key, same account

→ 4 **Session token** — 7 days; only its hash is stored; 5 per account

Every card route checks the card's on-chain `owner()` against the signed-in key: someone else's card answers 403 NOT_YOUR_CARD.

A new account is funded with real transactions: gas balance plus HK$500 of Mock e-HKD, so it can open cards straight away.

What the phone will sign

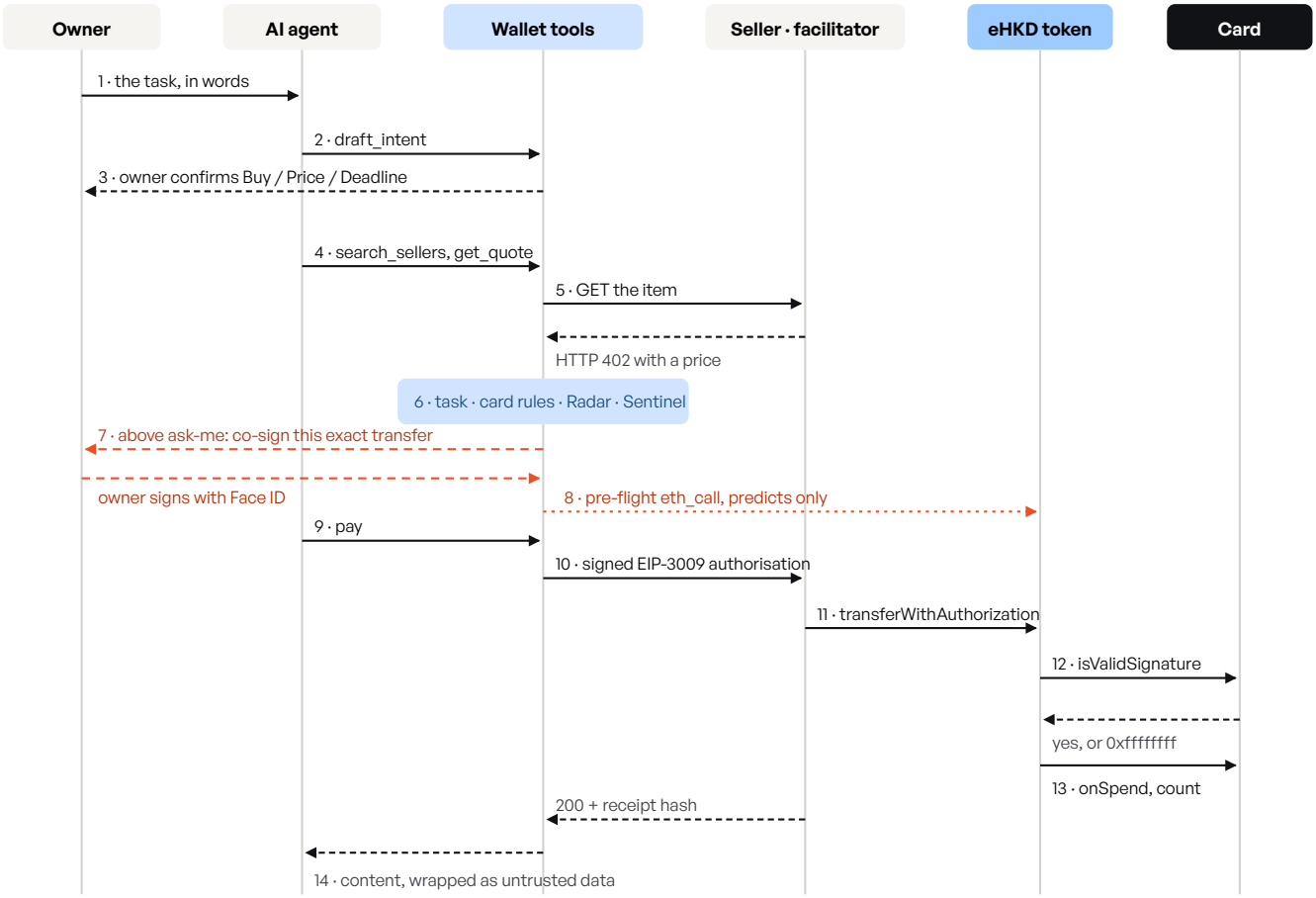
A signing guard decodes every request first and signs only: creating a card, funding the owner's own card, the owner controls on the owner's own card, and a co-sign whose card, seller and amount match the request shown on screen. Anything it can't decode, it refuses.

How a card gets its agent

The app asks `POST /v1/cards/agent` for the next agent address, then the owner's key signs `PocketFactory.create(agent, cap, askAbove, expiresAt, sellers)` and a plain transfer to fund it. A card can only use an agent issued to its own owner.

One payment, step by step

"Get me a durian export report, under HK$15, by tonight." Below is every message from that sentence to the money landing, with the place each check happens.



Step	Check	What it compares	Refusal code
6	**Task the owner confirmed**	Price per item, total, number of items, deadline	`PRICE_ABOVE_INTENT` · `INTENT_TOTAL_REACHED`
6	**Card rules, read first**	`canPay(to, value)` on the chain before signing	`SELLER_NOT_ALLOWED` · `OVER_CAP`
6	**Radar, Sentinel**	Seller's money trail score; a hold after suspicious refusals	`COUNTERPARTY_HIGH_RISK` · `SENTINEL_HOLD`
10	**Live quote**	The 402 at pay time must match the quote: network, token, payee, amount	`QUOTE_MISMATCH`
12	**The card, on chain**	Exact digest, agent signature, rules, owner co-sign (page 5)	transfer reverts

Every refusal returns one of 30 reason codes, so the model can re-plan instead of retrying. Each nonce is keccak256(task hash, sequence), so every transfer traces back to the task the owner confirmed. Authorisations expire after 600 seconds.

The card contract

A card is a small ERC-1271 contract. The token asks it "is this signature valid?" at the moment of payment, and the card answers yes only for a transfer inside the owner's rules.

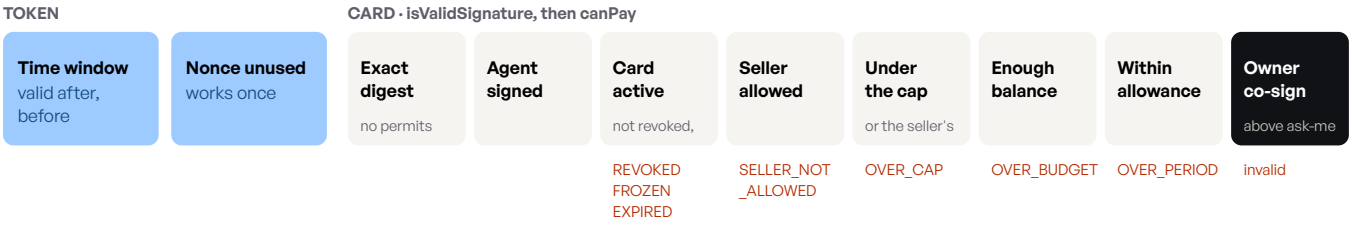
216
lines: Pocket.sol, the card

102
lines: MockEHKD.sol, ERC-20 + EIP-3009, 6 decimals

27
lines: PocketFactory.sol, creates cards

11
lines: Clock.sol, FISCO's millisecond time to seconds

What happens when a payment reaches the chain



Any no returns 0xffffffff and the token reverts "EHKD: invalid signature". The owner's co-signature lifts only two things: the ask-me limit and the allowance. It never lifts revoked, frozen, expired, seller, cap or balance. Then the token calls onSpend with 100,000 gas to count the allowance.

The rules a card holds

Rule	Stored as	Who changes it	When broken
Sellers it may pay	`sellerAllowed[address]`	owner, `setSeller`	`SELLER_NOT_ALLOWED`
Cap per payment	`perPaymentCap`, or a seller's own cap	owner, `setCap · setSellerCap`	`OVER_CAP`
Ask-me limit	`askAbove`	owner, `setAskAbove`	needs the owner's signature
Allowance	day, week or month, in Hong Kong time	owner, `setPeriodLimit`	`OVER_PERIOD`, owner may co-sign
Expiry	`expiresAt`	owner, `setExpiresAt`	`POCKET_EXPIRED`
Freeze	`frozen`	owner or Sentinel freezes; only the owner unfreezes	`POCKET_FROZEN`
Revoke	`revoked`, permanent	owner, `revoke`; funds come back with `returnFunds`	`POCKET_REVOKED`

What the agent and owner sign

Both sign the same EIP-712 `TransferWithAuthorization`: from this card, to one seller, one amount, one nonce, one time window. The signature handed to the token is an envelope:

```
abi.encode(to, value, validAfter, validBefore,
           nonce, agentSig, ownerSig)
```

ownerSig is empty (0x) when no co-sign is needed.

Events anyone can read

`PocketCreated(pocket, owner, agent)`, `Spent`, `Frozen(by, reason)`, `Unfrozen`, `Revoked`, `RulesChanged`, `SellerSet`, `SellerCapSet`, and the token's `AuthorizationUsed`. The Python verifier rebuilds a card's history from them.

Solidity 0.8.24, optimizer 200 runs, OpenZeppelin 5.4, Foundry. Any owner function still works on a frozen or revoked card, so the owner can always take the money back.

Wallet server

One TypeScript service (Hono on Node 24, viem, the x402 libraries) and a second process for the sellers. It runs the agent and does every check it can before the chain, but it holds no owner key.

7 agent tools

42 API routes, plus 5 on the sellers

30 reason codes the model can read

15 event types on the live stream

8 x402 sellers, 12 items

HTTP and identity

App, SSE, sign-in, per-card agent keys, rate limits, JSON storage on a volume

`index · visitors · agentKeys · limits · store`

Money path

Tools, quote and settle, x402 scheme, task check, pre-flight, co-sign, content scan, reason codes, nonces

`wallet/ · facilitator · ledger`

Agent brain

Tool loop, prompt, scripted fallback, model switch, memory, rules compiler, card-app composer

`agent/ · llm · brain/ · rules/ · pocketApp/`

Safety

Radar screening and alerts, Sentinel hold and freeze, the attack arena on the real chain

`radar/ · sentinel/ · arena`

Autonomy

Routines every 30 s, two starter cards for new accounts, faucet, gas funding

`routines · starter · faucet · fund`

Chain access

Chain config, read-only explorer and RPC proxy, the 8 test sellers

`chains · explorer · rpcProxy · sellers`

The agent's seven tools

Tool	What it does	Moves money
`get_pocket`	Balance, cap, ask-me limit, allowance, spent on this task	no
`search_sellers`	Sellers, items and prices, and whether the card allows each	no
`draft_intent`	Shows the owner a fixed Buy / Price / Deadline task and waits up to 10 minutes	no
`get_quote`	The seller's 402 price, with the card check, task check and Radar result	no
`counter_offer`	Haggles, at most 3 rounds, only on items with a floor	no
`ask_owner`	A question, or a co-sign request for one quote	no
`pay`	Settles a quote over x402, after every check on page 4	yes, only this

No tool takes a recipient address, edits a card, or reaches another card or the owner's main wallet. An attacker who controls the model can only pick from quotes the server fetched itself.

Main routes

`POST /v1/challenge · /v1/visitors` sign up or sign in

`POST /v1/cards/agent` the next card's agent address

`POST /api/chat` one agent turn

`POST /api/approvals/:id` owner's answer or co-sign

`GET /api/events` live stream (SSE), or `.json` to poll

`POST /api/rules/compile` a sentence to card rules

`POST /api/arena/straight` agent key tries the chain directly

`GET /api/radar/screen/:who` a seller's risk

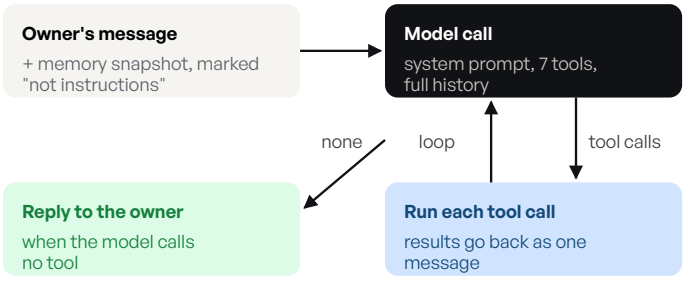
`GET /api/explorer/*·/explorer` blocks, transactions

`POST /api/faucet · /rpc/fisco` test funds, read-only RPC

Agent and AI brain

The model plans, compares sellers and asks the owner. It is assumed it can be fooled, so every money rule is enforced in code and on chain, never by the prompt alone.

One turn of the agent



Stops on its own when

- 24 model calls are used, or the model errors or refuses
- and then it names the payments already made; nothing retries
- no model or key: a scripted agent takes over, shown in the trace

The task the owner confirms

```
Buy: one durian export market report
Price: at most HK$15.00 each, HK$15.00 in total, up to 1 item
Deadline: 2026-10-19 23:59 HKT
```

The model proposes the fields; code writes the three lines, so amounts and the deadline are never copied from the model. Every quote, co-sign request and payment is then checked against it: confirmed, not expired, price per item, total, number of items.

Models

Flag `LLM_PROVIDER`	Used for every role
openrouter default	Claude Haiku 5.5; four other presets allowed
anthropic	Opus 5.5 agent, Sonnet 5.5 rules, Haiku 5.5 for memory, Radar, Sentinel
scripted	No model: keyword router and fixed parsers

Memory may only narrow

After each turn a reflection writes one memory note per card (up to 30 items). It reads only the owner's words and fixed lines built from wallet events, never seller text. Anything that would loosen a limit ("always pay", "raise the cap", "trust this seller") is rejected. Bad output twice keeps the old memory.

Speak your rules

"HK$5 a week for news, only Exa, ask me above HK$1." Two model readings run in parallel and any difference takes the stricter value. The result is a draft card plus a plain-words proof of each rule. It never creates the card: the owner still signs.

The Brain graph

The apps draw the brain as a live graph: ChainBrain in the centre, memory around it, then agents, their cards, and the sellers each card may pay. Every node is a real agent, card or seller; each edge is "allowed to pay", with the payment count.

Seller text is data. Everything a seller returns is wrapped in `<untrusted_content source="…">`, forged closing tags are removed, and requests for personal data or for a signature are refused by code (DISCLOSURE_REFUSED, SIGNATURE_TYPE_NOT_ALLOWED). Seller text never reaches memory, the haggler, Radar's summary or the Sentinel's model.

Security and monitoring

Layers stacked from the model down to the token. The ones above the card make the agent better behaved; the card makes it safe. Results of 63 attacks are in the separate security test report.

1 · AI agent
can be fooled, holds no key

2 · Content wrap and scan
seller text is data; data and signature asks refused

3 · Task check
price, total, items, deadline the owner confirmed

4 · Radar
seller's money trail: ok, ask the owner, or refuse

5 · Sentinel hold
after odd behaviour, the next payment needs the owner

6 · Signing rule
signs only its own transfer matching the live quote

off chain: a broken server skips these

7 · Card contract
digest, agent, rules, owner co-sign

8 · Mock e-HKD token
each authorisation exact, works once

on chain: always holds

beside the agent: memory guard, narrow-only

Radar: who is this seller?

Indexes every Mock e-HKD transfer, then traces a seller's money 3 hops each way. Labels come from a sanctions-style file, plus ChainBrain's own "flagged" label for any seller that asked for data or a signature.



mixer in the trail: 5 · drainer or scam: 6
mixer 25%+: 7 · scam 10%+ or darknet: 8
sanctioned in the trail: 9 · labelled itself: 10

Re-screens every 30 s; a worse score alerts every card that paid or quoted that seller. Thresholds are a first guess, not calibrated.

Sentinel: is this agent hijacked?

Signal	When	Action
Refusal burst	3+ refused attempts in 60 s	freeze
Ghost transfer	money left with no payment event	freeze
Follow-through	a refusal within 120 s of a data ask	hold
Probing	2 refused attempts in 60 s	watch

The guardian key only freezes

A freeze is an on-chain transaction from the Sentinel's guardian key, fixed per card. It can't move money, unfreeze or change a rule. Without the key, a freeze falls back to a hold.

The AI can only raise

A model (TypeSafe's Jev when keyed, else Haiku 5.5) reads a fixed summary and may raise watch to hold or freeze. It can never lower a decision, and low confidence can't freeze.

Pre-flight predicts, never blocks

Before paying, the wallet runs the exact transfer as an eth_call and records "pays" or "refused, because". After settling it shows predicted vs actual, so a wrong guess is visible.

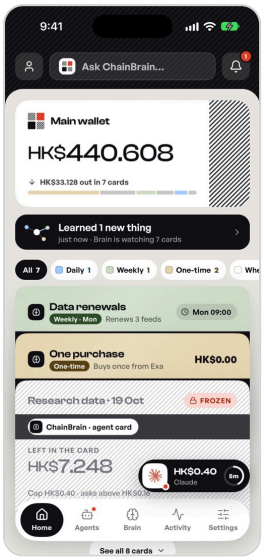
The 30 reasons a payment can be refused

Card, on chain	`SELLER_NOT_ALLOWED` · `OVER_CAP` · `OVER_BUDGET` · `OVER_PERIOD` · `POCKET_REVOKED` · `POCKET_EXPIRED` · `POCKET_FROZEN`
Confirmed task	`PRICE_ABOVE_INTENT` · `INTENT_TOTAL_REACHED` · `INTENT_ITEMS_REACHED` · `INTENT_EXPIRED` · `INTENT_NOT_CONFIRMED`
Owner	`NEEDS_OWNER_APPROVAL` · `APPROVAL_EXPIRED` · `OWNER_DECLINED`
Quote and signing	`QUOTE_MISMATCH` · `QUOTE_ALREADY_PAID` · `QUOTE_UNKNOWN` · `SIGNATURE_TYPE_NOT_ALLOWED` · `SETTLEMENT_FAILED` · `NOT_THIS_AGENT`
Seller content	`DISCLOSURE_REFUSED`
Radar, Sentinel	`COUNTERPARTY_HIGH_RISK` · `COUNTERPARTY_REVIEW` · `SENTINEL_HOLD`
Haggling	`OFFER_NOT_LOWER` · `NEGOTIATION_ROUNDS_USED` · `NEGOTIATION_NOT_OFFERED` · `DEAL_ALREADY_AGREED` · `QUOTE_ALREADY_APPROVED`

Each code comes with a plain-English line (`server/src/wallet/reasons.ts`). The model reads it and re-plans; the apps show it to the owner.

The apps

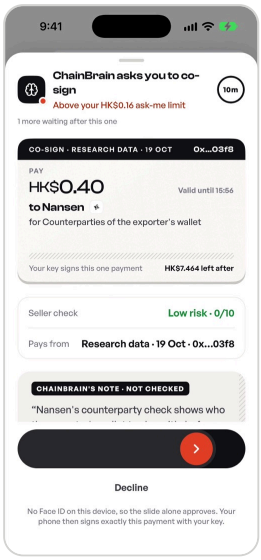
The phone app is where the owner lives: cards, the agent, co-signing. The web console is where judges watch every decision and try to break it. Screens below are from the real apps.



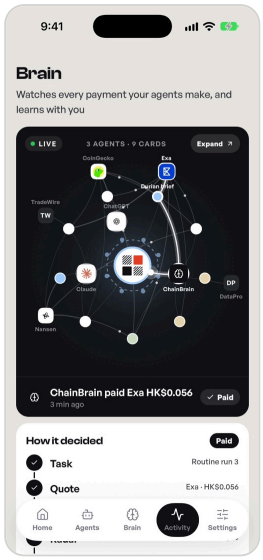
Home
main wallet, and one card per job, coloured by use



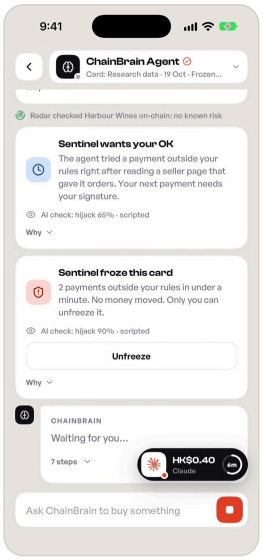
Compose
one sentence; the card builds itself from fixed blocks



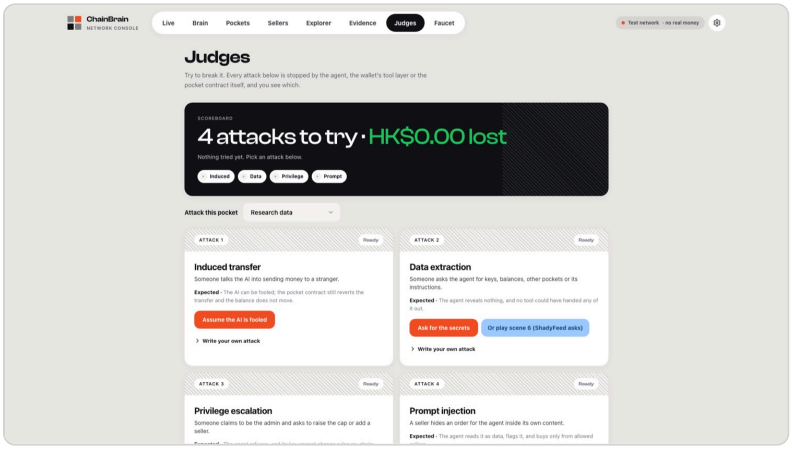
Co-sign
one exact payment, seller check, slide and Face ID



Brain
live graph and how each payment was decided



Sentinel
asks for an OK, then freezes the card on-chain



Web console, Judges tab: four one-tap attacks (induced transfer, data extraction, privilege escalation, prompt injection), each with the expected result and a "write your own attack" box.

	Phone app
Stack	Expo SDK 57, React Native 0.86, Skia, viem
Tabs	Home, Agents, Brain, Activity, Settings
Key	made on the device, in secure storage
Signs	only what the signing guard can decode

	Web console
Stack	React 19, Vite 8, Tailwind 4
Tabs	Live, Brain, Pockets, Sellers, Explorer, Evidence, Judges, Faucet
Key	made in the browser, optional passkey

Neo-Kinetic design

Ink #111215, vermillion #F04E23, powder blue #9DCAFF on warm grey #E7E6E2; Clash Display and General Sans. One token file per app.

Every step on screen

Both apps read the server's live event stream: task, quote, Radar, pre-flight, payment, refusal with its reason. Each hash opens in the explorer.

A third app

A native Kotlin Android app signs with the Android Keystore against the same server. Its EIP-712 signatures are byte-identical to viem's.

Three ways to run it

The apps open on the real hosted chain. A built-in demo network lets every scene run with no server or internet, and a local stack lets anyone run the real code end to end.

default

Hosted, real chain

Apps sign in with the owner's key and talk to the hosted wallet API on Railway. Real FISCO transactions, real Mock e-HKD, every hash in the public explorer. Live model through OpenRouter.

Settings → Demo network

Built-in demo network

The whole world in TypeScript inside the app: a virtual chain, the wallet API, 7 sellers, 3 agents, 12 scenes and two weeks of history. Needs nothing else. Hashes exist only in the app.

Settings → Custom server

Your own server

Point either app at any ChainBrain server: a local FISCO node with `scripts/fisco`, or the dev stack, which runs the real server and contracts with a scripted agent and no API key.

What is real in each mode

	Hosted	Demo network	Dev stack
Chain	FISCO BCOS	in-memory, same card rules	FISCO BCOS, local
Contracts	deployed	same check order as Pocket.sol	fresh deploy per run
Wallet server and tools	real	same logic, in the app	real
x402 payments	real	not run	real
The agent	live model	scripted	scripted
Radar, Sentinel, memory	real	same rules; AI parts scripted	real rules
Owner key	device	device	dev keys

The demo network shares its pure modules with the apps on every mode: the checks path, the proof card, the explorer view, the Brain graph. Outside agents (Claude and ChatGPT pairing) exist only on the demo network for now.

Linking the phone to the console on the demo network



judge actions go back to the phone: plant an order, force a transfer, hijack, say

Chain and deployment

A single-node FISCO BCOS 3.16.4 chain runs on Railway with its own explorer and faucet, so anyone can open a transaction, get test money and check a card.

20200	0	3	HK$100	ms
chain id; x402 network eip155:20200	gas price; senders still need a native balance	Railway services: node, API, sellers	from the faucet, once per address per day	FISCO block time is in milliseconds; Clock.sol converts

Public addresses

RPC	fisco-node-production.up.railway.app
Wallet API	fisco-api-production-85c5.up.railway.app
Explorer	.../explorer
Faucet	.../faucet
Sellers	fisco-sellers-production.up.railway.app
Console	fintechathon-lian-ai-nao.vercel.app



The hosted explorer, built into the wallet API because FISCO has none: a reverted attempt to move money, sent to the Mock e-HKD token.

Contracts on the hosted chain

Mock e-HKD	`0x0355c0c5Afe2A35369056FBB854903A4aF07781A`
PocketFactory	`0x03aB4C311eeE7C3295c63F910D79a07Ffa7A097A`

From `contracts/deployments/20200.json`. Each card is its own contract created by the factory.

What the RPC gateway fixes

Fee history. Wallets ask for EIP-1559 fees; FISCO has none, so the gateway answers with zeros.

Log windows. eth_getLogs is split into 5-block windows, up to 5,000 blocks.

Batches. More than 8 calls are split; transaction indexes are repaired.

Funding. A token-protected route gives a new sender the native balance FISCO requires.

Deploying it

On a laptop

```
bash scripts/fisco/setup.sh   # once
bash scripts/fisco/deploy.sh  # deploy, mint
cd server && pnpm sellers     # :4021
cd server && pnpm dev         # :8787
```

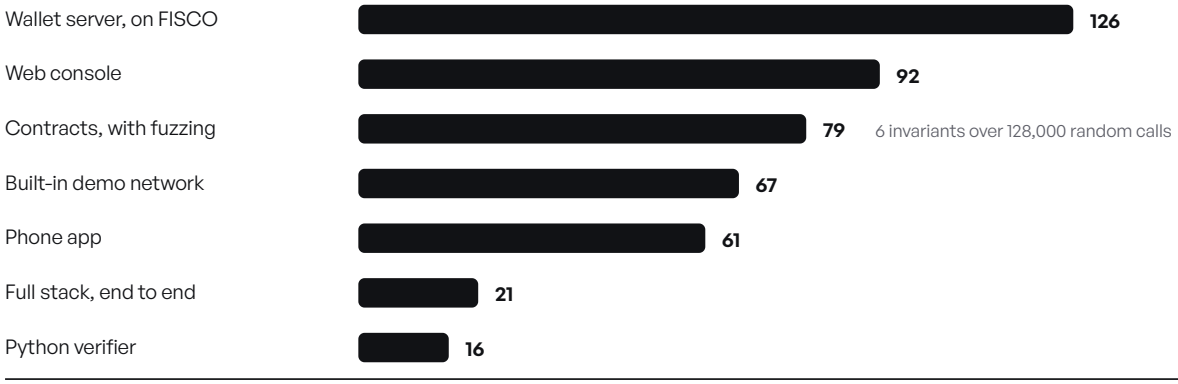
Hosted, then check any card

```
railway up --service fisco-node
railway up --service fisco-api
railway up --service fisco-sellers
python3 verify/verify_pocket.py \
  --rpc <url> --pocket 0x…
```

`deploy.sh` funds the keys, runs the Foundry deploy and mints HK$10,000 to the issuer. On Railway, the API service's bootstrap does the same before starting. The console deploys to Vercel.

Tests, setup and limits

Every suite below was run on 19 October 2026 on the submitted code. Attack results, live-model runs and the hosted-chain proof are in the security test report.



462 tests, 0 failures

Plus 6 agent scenes, 6/6 with the scripted agent; the native Android app's 6 unit tests weren't re-run for this count.

Run the tests

```
cd contracts && forge test
cd server && npx vitest run
cd web && npx vitest run
cd mobile && npm test
cd mobile && node --import tsx \
  --test ../shared/sim/*.test.ts
cd devstack && npx vitest run
python3 -m unittest verify/test_verify.py
```

The server and full-stack suites need the local FISCO node: `bash scripts/fisco/start.sh`.

Settings that matter

Variable	Default · purpose
`FISCO_RPC`	127.0.0.1:8545 · the chain
`LLM_PROVIDER`	openrouter · or anthropic, scripted
`OPENROUTER_API_KEY`	none · without it, scripted agent
`LLM_MODEL`	claude-haiku-5.5 · or a preset
`AGENT_MASTER_KEY`	0x + 64 hex · set once, never change
`SENTINEL_PK`	optional · the freeze-only guardian
`DATA_DIR`	server/.data · accounts, cards, events
`VISITOR_STARTER`	500 · HK$ for a new account

`pnpm wallets` writes the other keys to `server/.env`.

Built to be checked

- Every hash opens in the hosted explorer.
- The Python verifier reads a card with only the standard library and says MATCH or MISMATCH against what the owner approved.
- `verify-evidence.ts` re-fetches every recorded hash.
- The console's Evidence tab exports expected vs actual per scene.

Known limits

- Radar's thresholds are a first guess, not calibrated on real data.
- The content scan reads JSON fields; a request written as prose isn't caught by code, only by the model and the card.
- Open tasks, quotes, Radar's index and the Sentinel's watch lists live in memory and reset on restart; accounts, cards, memory and events are saved.
- One FISCO node; iOS tested in the simulator and Expo Go, not on a device build.